AI開発の現状概観 ![High]

- AI開発はPython中心だがTypeScript/Rustも台頭
- 用途ごとに最適言語が分化（モデル開発・Web統合・エージェント等）
- LLM普及でAPI連携・リアルタイム推論の要件増加
- 型安全性・パフォーマンス・エコシステムが選定指標
- 主要採用事例: Google, OpenAI, Microsoft, AWS

  出典: Python公式, Gartner, PyTorch公式

言語別AI対応度マトリクス ![High]

言語	AIライブラリ	LLM統合	型安全性	性能	エコシステム
Python	高	高	低	中	非常に高
TypeScript	中	高	高	中	高
Rust	中	中	非常に高	高	中

- PythonはAI/LLMで圧倒的
- TypeScriptはWeb/クラウド統合で急成長
- Rustは高性能・安全性で注目

出典: PyTorch公式, The New Stack, Stack Overflow調査

Pythonの強みと課題 ![High]

- AI/MLライブラリ・LLM統合で圧倒的優位
- 豊富な公式・サードパーティ資源
- 型安全性・実行速度・保守性に課題
- 歴史的偶発性とコミュニティ効果が普及要因
- 研究・実験・PoC層でのデファクト

  出典: PyTorch公式, TensorFlow公式, 論文

TypeScriptの台頭 ![Medium]

- Web/クラウドAI統合で急速に普及
- 型安全なエージェント開発に強み
- LLM統合フレームワーク（LangChain.js等）が充実
- Microsoft, Vercel等で採用拡大
- Pythonとの二層構造が主流

出典: TypeScript公式, LangChain.js, The New Stack

Rustの役割と導入戦略 ![Medium]

- 高パフォーマンス・安全性が最大の魅力
- AI推論エンジン・分散システムで採用増
- 学習コスト・エコシステム成熟度は課題
- 低レイテンシ/高負荷領域で段階導入が現実的
- HuggingFace, AWS等でPoC進行

  出典: Rust公式, llm-rs, Stack Overflow調査

LLM統合フレームワーク比較 ![Medium]

- LangChain/LlamaIndexがPython・TypeScript両対応
- フレームワーク依存をアダプタ層で隔離推奨
- 交換可能設計で将来の技術進化に備える
- API連携・RAG・エージェント開発が容易
- 固定採用より柔軟な設計が重要

出典: LangChain公式, LlamaIndex公式, O’Reilly調査

エージェント開発ツールの動向 ![Medium]

- Python: LangChain, LlamaIndex, Haystack等
- TypeScript: LangChain.js, LlamaIndex.js等
- Rust: llm-rs, burn等（発展途上）
- Web統合・UIはTypeScriptが主流
- PoC/高性能領域はRustも選択肢

出典: 各公式ドキュメント, 業界調査

実装パターンカタログ ![Medium]

- Python: AIコア・推論API・バッチ処理
- TypeScript: Web UI・APIゲートウェイ・RAG統合
- Rust: 高速推論・分散処理・エッジAI
- LLM統合: Adapter/Facadeパターン推奨
- ポリグロット連携: gRPC/REST/Message Queue活用

  出典: 各言語公式, フレームワーク事例

採用ロードマップ ![Medium]

- 直近2年は「Python＋TypeScript」二層構造を標準化
- RustはPoC・高負荷領域で段階導入
- LLMフレームワークは交換可能設計で導入
- チーム内で用途別に言語分担を明確化
- 継続的な技術ウォッチとPoC評価を推奨

出典: 各公式, O’Reilly, Stack Overflow調査

リスク・注意点

section: alert

- Python依存の過大評価リスク
- LLMフレームワークの進化速度に注意
- Rust導入は学習コスト・人材確保が課題
- 自己参照バイアスや未検証ソースに注意
- 技術選定は常に再評価が必要

推奨アクション

section: success

- Python（AIコア）＋TypeScript（統合/UI）を基本構成に
- LLMフレームワークはアダプタ層で柔軟に
- RustはPoC・高負荷領域で段階的に評価
- チームで用途別言語戦略を明確化
- 継続的な技術調査・PoCを実施

参考図解

- 三重力圏モデル：Python（AIコア）・TypeScript（統合）・Rust（高負荷）
- 用途別の最適言語分布を可視化

まとめ

section: dark

- AI時代の開発言語は「用途別ポリグロット」が現実解
- Python＋TypeScriptの二層構造が直近最適
- Rustは高負荷領域での将来有望株
- 技術選定は継続的な見直しが必須
- フルスタックエンジニアは複数言語の実践力が求められる

関係図

Matrix

Mindmap

Thank You